

# ROS 2 LAB #1: GETTING STARTED GUIDE

Abdelbacet Mhamdi  
Senior-lecturer, Dept. of EE  
ISET Bizerte — Tunisia  
✉ a-mhamdi

**Abstract** — This guide introduces the fundamentals of ROS 2 (Robot Operating System 2) using the TurtleSim simulator. TurtleSim is a simple 2D simulator that helps beginners understand core ROS 2 concepts without the complexity of real hardware.

## I. ENVIRONMENT SETUP

### A. Checking The Shell

First, identify which shell we're currently using:

```
echo $SHELL
```

This prints the path to the current shell executable (bash, zsh, etc.). Different shells may require different setup procedures, so this information is important for proper configuration.

### B. Sourcing ROS 2 Installation

Add ROS 2 commands and environment variables to the current terminal session:

```
source /opt/ros/humble/setup.bash
```

This step is essential as **ROS 2** commands won't be available without sourcing. The sourcing must be done in every new terminal session (each new terminal needs: `source /opt/ros/humble/setup.bash`), or we can add this line to our shell's configuration file (`.bashrc`, `.zshrc`) to make it automatic.

## II. RUNNING OUR FIRST ROS 2 NODE

*Reminder: In any new terminal, run `source /opt/ros/humble/setup.bash` before ROS 2 commands.*

A **node** is a fundamental unit of computation. Each node is a separate process that performs a specific task. Nodes communicate with each other through topics, services, and actions. The command structure follows the pattern:

```
ros2 run <package_name> <executable_name>
```

To list all executables in a package:

```
ros2 pkg executables <package_name>
```

The following command lists all executables in the turtlesim package:

```
ros2 pkg executables turtlesim
```

Let's launch the **TurtleSim** simulator:

```
ros2 run turtlesim turtlesim_node
```

This command opens a blue window with a turtle in the center.

## III. UNDERSTANDING TOPICS AND COMMUNICATION

### A. Listing Active Topics

**Topics** are named communication channels where nodes can publish (*send*) or subscribe (*receive*) messages. Display all currently active topics with their message types:

```
ros2 topic list -t
```

The `-t` flag shows the message type for each topic. Expected output includes topics such as:

- `/turtle1/cmd_vel` [geometry\_msgs/msg/Twist]
- `/turtle1/color_sensor` [turtlesim/msg/Color]
- `/turtle1/pose` [turtlesim/msg/Pose]

### B. Inspecting Message Structure

To understand the data structure of messages used in topics, we can inspect a specific message type. This command shows the Twist message structure:

```
ros2 interface show geometry_msgs/msg/Twist
```

The Twist message contains linear and angular velocity components for 3D space.

```
Vector3 linear
float64 x
float64 y
float64 z
```

```
Vector3 angular
float64 x
float64 y
float64 z
```

For the 2D turtle, we primarily use `linear.x` (*forward/backward*) and `angular.z` (*rotation*).

#### IV. PUBLISHING MESSAGES

*Reminder: In any new terminal, run `source /opt/ros/humble/setup.bash` before ROS 2 commands.*

##### A. Single Movement Command

If we want to move the turtle forward by 1 unit, we can publish a single message to the `/turtle1/cmd_vel` topic:

```
ros2 topic pub --once /turtle1/cmd_vel
geometry_msgs/msg/Twist "{linear: {x: 1.0}}"
```

Let's breakdown the structure of the previous instruction:

- `ros2 topic pub`: Command to publish to a topic
- `--once`: Publish only one message then exit
- `/turtle1/cmd_vel`: The target topic name
- `geometry_msgs/msg/Twist`: The message type
- `"{linear: {x: 1.0}}"`: Message data in YAML format

##### B. Continuous Movement Command

Now, we want to publish velocity commands continuously to create circular motion:

```
ros2 topic pub -r 1 /turtle1/cmd_vel
geometry_msgs/msg/Twist "{linear: {x: 1.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: .7}}"
```

The key parameters are:

- `-r 1`: Publish at a 1 Hz rate (*i.e., 1 message per sec*)
- `linear: {x: 1.0}`: Move forward at 1 unit/sec
- `angular: {z: .7}`: Rotate at 0.7 radians/sec

As it can be observed, the combination of forward motion and rotation creates circular movement.

#### V. MONITORING COMMUNICATION

In **ROS 2**, it is crucial to monitor messages being published on a topic in real-time. This can be done using the `ros2 topic echo` command:

```
ros2 topic echo /turtle1/cmd_vel
```

This debugging tool displays all messages on the specified topic. When used alongside the continuous movement command, we'll see the velocity messages being printed continuously.

#### VI. INTERACTIVE CONTROL

*Reminder: In any new terminal, run `source /opt/ros/humble/setup.bash` before ROS 2 commands.*

##### A. Direct Turtle Teleop

Let's use TurtleSim's built-in keyboard control to teleoperate the turtle:

```
ros2 run turtlesim turtle_teleop_key
```

This executable is specifically designed for TurtleSim and publishes directly to `/turtle1/cmd_vel`, eliminating the need for topic remapping.

##### B. Keyboard Teleop with Remapping

In general, it is possible to control the turtle using keyboard input with topic remapping:

```
ros2 run teleop_twist_keyboard
teleop_twist_keyboard --ros-args --remap
cmd_vel:=/turtle1/cmd_vel
```

The command components are:

- `teleop_twist_keyboard`: Package and executable name
- `--ros-args --remap`: ROS 2 argument for topic remapping
- `cmd_vel:=/turtle1/cmd_vel`: Maps the default `cmd_vel` topic to `/turtle1/cmd_vel`

#### VII. SERVICES

Services are a way for nodes to request specific actions from other nodes. They are typically used for short-lived operations that require a response. It is possible to check the list of available services using the `ros2 service list` command. The type of a particular service can be determined using the `ros2 service type <service_name>` command. The command to call the service is `ros2 service call <service_name> <service_type> <arguments>`.

##### A. Absolute Position Control

It allows a node to set the exact position of another node. This is useful for precise movements or when a node needs to know its exact location. The command to call the service is `ros2 service call <service_name> <service_type> <arguments>`.

For instance, to teleport the turtle to the position (5.0, 5.0) with an orientation of 0.0 radians, you can use the following command:

```
ros2 service call /turtle1/teleport_absolute
turtlesim/srv/TeleportAbsolute "{x: 5.0, y: 5.0,
theta: 0.0}"
```

B. Relative Position Control

It allows a node to move another node relative to its current position. This is useful for incremental movements or when a node needs to move a certain distance in a specific direction.

For instance, to move the turtle forward by 2.0 units and rotate it by  $-2.0$  radians, you can use the following command:

```
ros2 service call /turtle1/teleport_relative
  turtlesim/srv/TeleportRelative "{linear: 2.0,
  angular: -2.0}"
```

VIII. SYSTEM VISUALIZATION

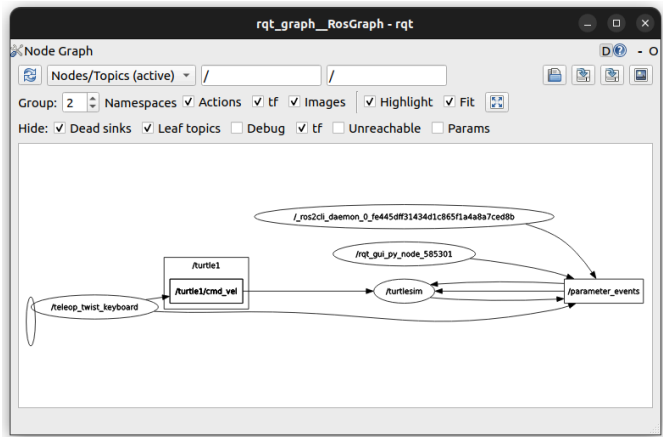
Reminder: In any new terminal, run `source /opt/ros/humble/setup.bash` before ROS 2 commands. There is a tool to visualize the relationships between nodes and topics, called `rqt_graph`:

```
rqt_graph
```

This opens a graphical tool displaying:

- Nodes as ovals
- Topics as rectangles
- Arrows showing communication direction

The visualization helps understand data flow in ROS 2 systems and is invaluable for debugging communication issues in complex robotic applications.



IX. SUMMARY

A. Terminology

| Concept     | Description                                      |
|-------------|--------------------------------------------------|
| Nodes       | Independent processes performing specific tasks  |
| Topics      | Named communication channels for message passing |
| Messages    | Structured data types (e.g., Twist for velocity) |
| Publishing  | Sending messages to topics                       |
| Subscribing | Receiving messages from topics                   |
| Packages    | Collections of related nodes and resources       |
| Remapping   | Redirecting topic names for system flexibility   |

B. Command Reference

| Command                                                             | Purpose                       |
|---------------------------------------------------------------------|-------------------------------|
| <code>which \$SHELL</code>                                          | Check current shell           |
| <code>source /opt/ros/humble/setup.bash</code>                      | Source ROS 2 environment      |
| <code>ros2 run &lt;package&gt; &lt;executable&gt;</code>            | Run a ROS 2 node              |
| <code>ros2 topic list -t</code>                                     | List active topics with types |
| <code>ros2 interface show &lt;msg_type&gt;</code>                   | Show message structure        |
| <code>ros2 topic pub &lt;topic&gt; &lt;type&gt; &lt;data&gt;</code> | Publish to topic              |
| <code>ros2 topic echo &lt;topic&gt;</code>                          | Monitor topic messages        |
| <code>rqt_graph</code>                                              | Visualize system graph        |

### Task 1: Fibonacci Spiral

Implement a **ROS 2** application using the `turtlesim` package to visualize the Fibonacci spiral. The turtle should trace the golden spiral pattern governed by the mathematical equation:

$$r = a \times \Phi^{\frac{\theta}{2\pi}} \quad (1)$$

where:

$r$  radial distance from the origin,  
 $a$  a constant that controls the size of the spiral,  
 $\Phi$  the golden ratio (approximately 1.618),  
 $\theta$  the angle in radians.



You can choose  $\theta$  ranging from 0 to  $\frac{\pi}{4}$  with small increments. The resulting spiral should smoothly transition as the turtle moves, creating a visually appealing pattern that looks like the image shown hereafter.

